



**SCHOOL OF COMPUTER SCIENCES
UNIVERSITI SAINS MALAYSIA**

CPC354: COMPUTER GRAPHICS & VISUALIZATION

ASSIGNMENT 2

SEMESTER 1 2025/2026

School of Computer Sciences, Universiti Sains Malaysia
11800 USM, Penang, Malaysia

NAME	MATRIC NO.	E-MAIL ADDRESS
AHMED H.M. TAHA	22304588	ahmedtaha@student.usm.my
ZHOU YIANG	22305072	evanzhou@student.usm.my
TAN SHAN HANG	22303933	tanshanhang2004@student.usm.my
SHIRWA MOHAMED ABDULLAHI	22304903	shirwa.ma@student.usm.my

Submission date: 18th January 2026

Table of Contents

1.0 TASK CONTRIBUTION.....	2
2.0 INTRODUCTION.....	4
3.0 LITERATURE REVIEW.....	6
4.0 PROPOSED METHOD.....	8
4.1 Hierarchical Structure of the Robot Arm.....	8
4.2 Transformation sequence for each joint.....	9
4.3 Finding the tip of the gripper.....	9
4.4 Gripper design.....	10
4.5 Picking and Placement.....	10
4.6 Visual Architecture & Layout.....	11
4.8 Control Mechanisms.....	12
4.9 Real-Time Status Feedback.....	12
5.0 AI & EXTERNAL CODE DISCLOSURE.....	13
5.1 Generative AI Tools Used.....	13
5.2 AI Usage & Refinement Process.....	13
Example A: Procedural Geometry Generation.....	13
Example B: Advanced Lighting Shader.....	14
Example C: Dashboard Control Logic.....	15
5.3 Critical Analysis & Intellectual Contribution.....	16
5.4 External Code Citations.....	16
6.0 CONCLUSION.....	17
6.1 Success Factors and Validation.....	17
6.2 Limitations and Challenges.....	18
6.3 Future Improvements.....	18
7.0 REFERENCES.....	19

1.0 TASK CONTRIBUTION

TEAM MEMBER	CONTRIBUTION
AHMED H.M. TAHA	• Designed and implemented the control panel UI for interactive parameter adjustment. • Implemented slider logic with real-time updates and value constraints. • Handled keyboard input for precise control of transformations. • Integrated a colour picker for dynamic visual customization.
ZHOU YIANG	• Implemented fragment shaders including Phong, diffuse, specular, and rim lighting to enhance visual realism. • Performed lighting calculations using surface normals, light direction, and view vectors. • Implemented hierarchical rendering in render() to correctly propagate transformations. • Created procedural geometry using CreateCylinder() for flexible and well-lit 3D components.
TAN SHAN HANG	• Designed the state machine to manage animation flow. • Defined keyframe poses using getPoses(). • Implemented pose interpolation via lerpPose() for smooth transitions. • Controlled animation behavior using animate(),

	easing functions, and loop toggling.
SHIRWA ABDULLAHI	• Implemented gripper control logic for object manipulation. • Applied forward kinematics to compute joint and end-effector transformations. • Developed object attachment calculations to ensure consistent grasping behavior.

2.0 INTRODUCTION

The “Pick and Place” system, using a robotic arm primarily as the main component, is a simple functioning and programmable system that is widely used in manufacturing and logistics. These systems are automated to do repetitive tasks on a daily basis, for example relocating or categorizing mechanical parts, to keep things moving without pause thus ensuring productivity. Currently in modern days, the “Pick and Place” system is a more efficient tool, optimized with AI-powered support, and has the ability to learn and improve from previous errors [1].

Despite the beneficial factors, when it comes to practically setting up and deploying the physical robotic arms, there are several technical challenges. Precision in rigorous movements is one of them. In certain industrial tasks, any inaccuracies or miscalculations in positioning may lead to property damage. Furthermore, maintaining systems precision in high-speed environments limits their effectiveness. Another challenge is the setup complexity. This makes maintenance and problem troubleshooting harder to be implemented. Regardless, any challenges would pose costly operational downtimes to industries [2].

To ensure efficient and safe hardware operation, the Digital Twin concept is proposed. A digital twin is a virtual simulation of a physical object or system that reflects the appearance, behavior, and performance of its real-world counterpart. The exchange of data between the actual robotic arm and the digital twin enables real-time monitoring and analysis. This helps us to easily visualize the actual environment and dynamically control the robotic arm in immersive ways. Thus, we will get more insights on how the robotic arm functions and interacts with the real world [3]. In the field of computer graphics and visualization, we can utilize WebGL to create a browser that allows access to the digital twin representation.

With the implementation of Digital Twin concept on the “Pick and Place” system, a safe sandbox is provided for developers. The main benefits of using a digital twin is that it allows development teams to predict and validate performance in a virtual environment, hence they can be informed with the most current data, thus making optimal decisions in full context. Furthermore, by connecting digital twins to their counterparts, teams can also remotely monitor operations in real time to identify, analyze and resolve issues. AI can also be infused into computer vision models to detect system defects in the real world. Lastly, digital twins allowed companies to test and

verify AI models in the simulation before deploying the models in the real world. This reduces physical prototyping, resulting in cost savings throughout a company [4].

In short, the combination of web technology, alongside real-time rendering mechanical engineering allows a more efficient and safer “Pick and Place” system to be deployed and maintained. The power of digital twins shall highly address the inherent risks of system operations to ensure a safer and more efficient industrial environment.

3.0 LITERATURE REVIEW

Robot arm simulations and articulated models are an important topic in computer graphics and robotics. Robot arms are often used as examples of articulated models because they have clear joint structures, hierarchical relationships, and strong connections to real-world systems [6]. Simulating robot arms in a virtual environment allows users to observe motion, test control methods, and study interaction behavior without using physical hardware, which reduces cost and risk [8]. Over time, robot arm simulations have developed from simple visual models into more advanced systems that combine hierarchical modeling, animation, kinematics, and real-time interaction [7].

Early robot arm simulation systems mainly focused on showing the structure and movement of mechanical parts. These systems were designed to help engineers understand robot motion using graphics instead of only equations or text descriptions [8]. By visualizing joint angles and movement paths, engineers could test ideas before applying them to real robots. Most early systems relied on forward kinematics, where joint values are applied one by one along the arm to calculate the final position of the end effector [9]. This idea became the basic model for representing robot arms as a chain of connected rigid links.

Hierarchical modeling later became a key technique for representing robot arms in computer graphics. In a hierarchical model, each part of the robot is connected in a parent–child structure, so when a parent part moves, all child parts move with it [7]. This structure matches how real robot arms work, where motion at the base affects the entire arm. Scene graphs and hierarchical transformation trees are commonly used to manage these relationships, making it easier to apply transformations and organize complex models [7]. Research on articulated figures also shows that hierarchical structures are important for controlling and animating jointed models in a clear and systematic way [6].

Animation is essential in robot arm simulation because it shows how the robot moves over time. One common animation method is keyframe animation, where several important poses are defined and the system generates smooth motion between them [6]. This method is popular in real-time applications because it is simple and easy to control. However, interpolating rotations can be difficult. Using Euler angles may cause problems such as gimbal lock or unnatural motion. To solve this, quaternion-based rotation was introduced, allowing smooth and stable

interpolation of rotations [5]. Quaternion interpolation has since become a standard technique for animating joint rotations in articulated models like robot arms.

Easing functions are often added to animation systems to improve motion quality. Easing controls how fast motion starts and stops, making movement appear smoother and more natural. This is especially useful for robot arm simulations, where sudden changes in speed can look unrealistic or confusing. These animation techniques are now widely used in articulated animation systems [6].

Interaction is another important part of modern robot arm simulations. Instead of only showing predefined animations, interactive systems allow users to change joint angles, start or stop animations, and modify visual settings in real time [7]. User interfaces such as sliders and control panels provide an easy way to adjust parameters, while keyboard input allows more precise control. In most graphics systems, interaction works by changing values in the hierarchical model, which are then updated and rendered in the next frame [7]. Early robot simulation systems already highlighted the importance of interaction for testing and visualization [8], and this idea is still central in current applications.

Modern robot arm simulations combine hierarchical modeling, kinematics, animation, and rendering into a single system. Hierarchical modeling defines how parts are connected, kinematics determines positions and orientations, animation controls how these values change over time, and rendering displays the final result on the screen [6][7]. Lighting techniques such as Phong shading are often used to improve visual clarity. Although lighting does not affect the correctness of motion, it helps users better understand the shape and movement of the robot arm, especially in interactive environments [7].

In summary, robot arm simulations in computer graphics have evolved from simple visualization tools into interactive and animated systems. Hierarchical modeling provides a clear structure for articulated models [7], keyframe animation with quaternion interpolation ensures smooth motion [5][6], and interactive control allows users to explore and understand robot behavior in real time [8]. These ideas form the theoretical background for the robot arm simulation implemented in this project.

4.0 PROPOSED METHOD

4.1 Hierarchical Structure of the Robot Arm

The robot arm that is implemented in our code is modelled as a kinematic chain, which is also known as a scene graph. In this system, every part of the arm is attached to the part that comes before it. If the 'parent' moves, then the 'child' will move as a result.

We used a global state called 'modelViewMatrix'. This holds all the position and rotation data as the code draws the arm from the bottom to the top.

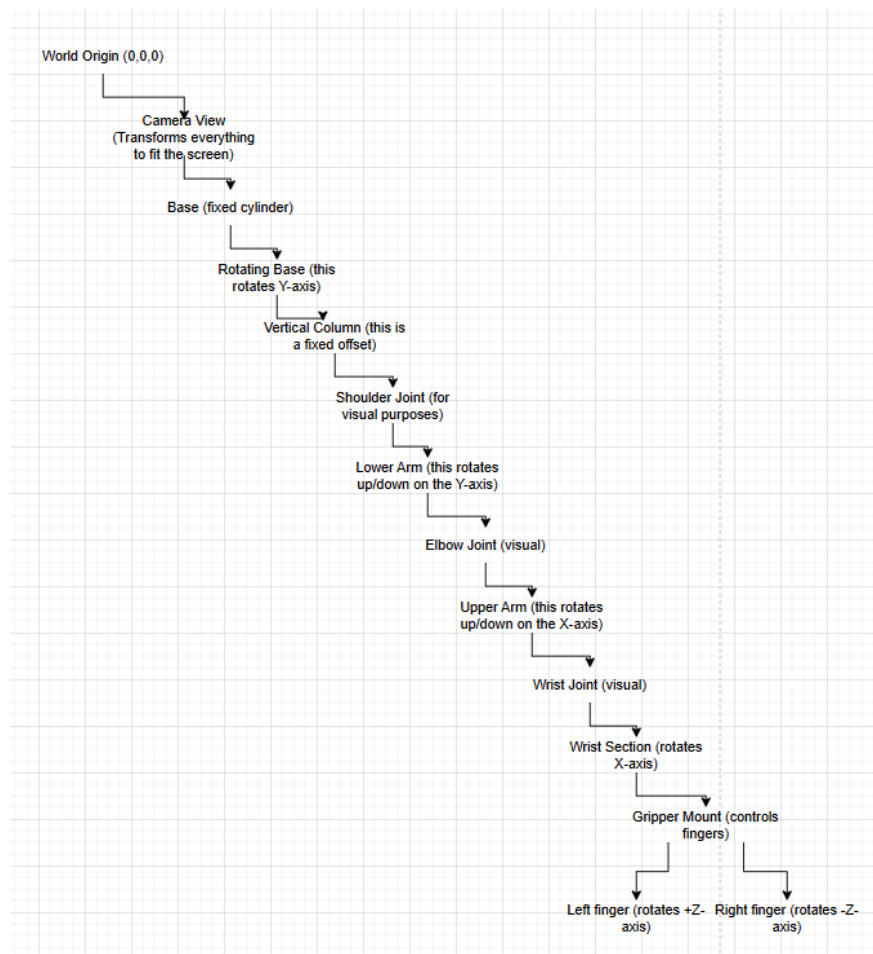


Figure 1 - Diagram showing hierarchy of robot arm

4.2 Transformation sequence for each joint

To determine what the final position of any part of the arm will be, we used matrix multiplication. The position of a specific vertex (v) is calculated by taking the camera vertex (V) and multiplying it by the accumulated model matrices (M) for every joint that goes up to it.

$$v_{Final} = V \times M_{base} \times M_{lower} \times M_{upper} \times \dots \times v_{Local}$$

The sequence of transformations that are applied are:

1. Base rotation: Translate up to the platform height and apply the Y-axis rotation matrix (R_y)
2. Lower Arm : We then move the vertical column, and then apply an X-axis rotation (R_x)
3. Upper Arm: Then we move along the length of the lower arm, and then apply another X-axis rotation
4. Wrist: We move along the upper arm, and then apply one final X-axis rotation

To make all these operations happen, the code uses 4x4 matrices. For example, the rotation matrix for moving the joints up and down on the X axis is defined as:

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4.3 Finding the tip of the gripper

To pick up any object it finds, the system needs to know exactly where the tip of the gripper is in the 3D world. We cannot use the drawing matrices for this because they include the camera's perspective.

Instead, there is a separate function called 'computeGripperWorldPosition()'. This function uses the same calculation as the drawing loop, but it removes the camera view.

$$Position_{tip} = T_{base} \times R_{base} \times T_{arm1} \times R_{arm1} \times T_{arm2} \times R_{arm2} \times \dots$$

The result of this is a coordinate in three dimensions that represents the absolute real-world location of the gripper, which is used to check distance to the cube.

4.4 Gripper design

The gripper uses two jaws that are parallel to one another to pick things up. They are two 'fingers' that the jaw has to rotate around the Z-axis. The left finger goes in the positive direction, while the right goes in the negative direction.

The gripper also changes size based on the object. If the user makes the cube larger using the slider (objectScale), the fingers will automatically lengthen and widen so they do not go through the object. This is shown by the formula:

$$FingerSize = 1.0 + (ObjectSize - 1.0) \times 0.3$$

4.5 Picking and Placement

The logic for this part is controlled by a Finite State Machine (FSM). This means the robot arm is always in one of eight states, and smoothly transitions between them using an easing function ($2t^2$) to make the movement look more realistic.

The steps are:

1. Face Pick: Rotate the base to align with the blue platform
2. Reach: Lower the arm
3. Grab: Close the gripper
4. Lift: Raise the arm
5. Face Drop: Rotate 180° to the green platform.
6. Lower: Descend.
7. Release: Open the gripper.
8. Return: Reset to centre.

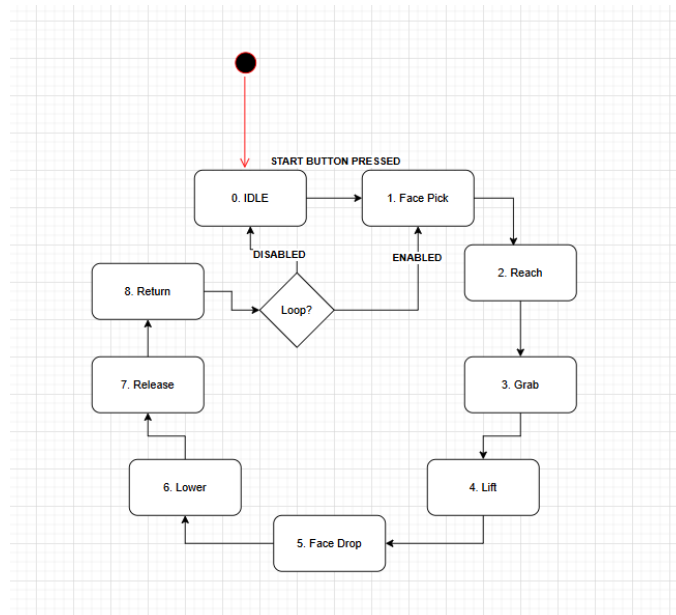


Figure 2: Diagram showing Finite State Machine for Picking and Placement

To make the picking more simplified, we just use a distance-based attachment system. The steps are:

1. Check the distance between the tip of the gripper and the centre of the cube.

$$Distance = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

2. If the distance < 0.4, then the variable 'objectAttached' becomes true.
3. Once it is attached, the code will stop drawing the cube at its own coordinates and will just use the gripper's coordinate matrix to draw it instead. This makes the cube stick to the gripper as if they are one object.

4.6 Visual Architecture & Layout

The interface is designed with a darker layout and aesthetic to minimize visual fatigue and highlight the 3D canvas. We did this using CSS3 linear gradients (#0a0a12 to #1a1a2e) and flexible box layouts (display: flex) to ensure the control panel and WebGL canvas adapt to different screen sizes.

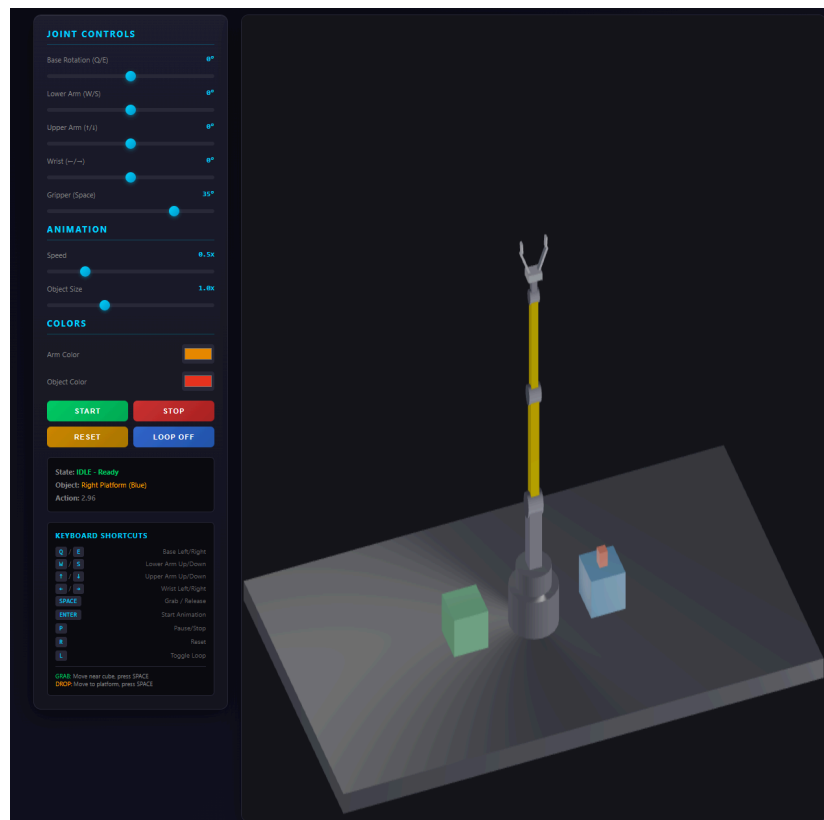


Figure 3: showing design of Robotic Arm UI

4.8 Control Mechanisms

The UI uses two different modes of interaction that are joined by an event-driven architecture:

- GUI Sliders: We used Range inputs to allow for more direct manipulation of the forward kinematics. Each slider uses a specific index in the `theta[]` array, such as `slider1` controlling `theta[1]`. The `oninput` event will then trigger the re-rendering.
- Keyboard Shortcuts: We used `keydown` event listeners to allow users to use the keyboard to make specific adjustments. This allows for more continuous joint rotation.

4.9 Real-Time Status Feedback

The system also uses a dynamic status monitor that contains:

- State Display: The current animation phase is shown based on the active index of the Finite State Machine
- Proximity Sensor: The UI then calculates the Euclidean distance between the gripper and the object in real-time. It then changes the text colour to green when the gripper is within a valid range of the object (in our case, <0.4 units)

5.0 AI & EXTERNAL CODE DISCLOSURE

As AI usage has become commonplace in the tech landscape today, we have also utilised it to help with development of our assignment. The section below will highlight some of the tools used, responses, and modifications we made to ensure we understood the code while still keeping development fast.

5.1 Generative AI Tools Used

- ChatGPT (OpenAI, GPT-4o): Used for conceptualizing the maths for 3D primitives, debugging shader syntax errors, and generating CSS boilerplates for the user interface.
- GitHub Copilot: Used for autocomplete suggestions within JavaScript loops (e.g., generating arrays for vertex normals) and fast-tracking boilerplate HTML structures.
- Gemini 3 Pro - Used for debugging and generating the functions that were used inside the JS, as well as for understanding the given sample code and the project instructions.

5.2 AI Usage & Refinement Process

Below are three specific examples of how we used Generative AI to accelerate development, detailing the initial outputs, the specific research sources used to validate them, and the engineering process we used to refine the final code.

Example A: Procedural Geometry Generation

- Context: We needed a function to generate a cylinder for the robot's joints.
- My Prompt: "Write a JavaScript function to generate vertices for a 3D cylinder in WebGL. Make sure it can be used with the sample code I have provided, and is in a similar format."
- The AI Output: The AI provided a basic loop that created a continuous "tube" but used shared normals for both the sides and the top caps.

```
slider.addEventListener('change', (e) => {  
    variable = e.target.value;  
});
```

- Observations & Research:
 - Issue: Upon rendering, the lighting on the flat top caps looked puffy and rounded rather than flat. It wasn't very realistic.
 - Research: We searched for "WebGL hard edge lighting" and found a discussion on Stack Overflow titled "Does flat shading require this much vertex duplication? [10]" which explained that a single vertex cannot have multiple normals. To achieve a hard 90-degree edge, the vertices at the seam must be duplicated.

- The Final Implementation: Based on this finding, we rewrote the AI's loop to explicitly separate the geometry into "Side" and "Cap" sections, manually assigning hard-coded normals (0,1,0) for the top caps

```
// Top cap
positions.push(0, 0.5, 0, x1, 0.5, z1, x2, 0.5, z2);
normals.push(0, 1, 0, 0, 1, 0, 0, 1, 0);

// Bottom cap
positions.push(0, -0.5, 0, x2, -0.5, z2, x1, -0.5, z1);
normals.push(0, -1, 0, 0, -1, 0, 0, -1, 0);
```

Example B: Advanced Lighting Shader

- Context: We wanted the robot to look metallic, not plastic.
- My Prompt: *"How do I calculate metallic specular highlights in a fragment shader?"*
- The AI Output (Complex): The AI suggested a standard PBR (Physically Based Rendering) workflow that needs texture maps.

```
float roughness = texture(uRoughnessMap, vTexCoord).r;
float metallic = texture(uMetallicMap, vTexCoord).r;
vec3 F0 = mix(vec3(0.04), albedo, metallic);
```

- Observations & Research:
 - Issue: The AI's code required loading external textures, which was outside the project scope.
 - Research: We searched for "simple metallic shader without textures" and found a thread on GameDev Stack Exchange titled "Realistic metal shader" [11]. The top answer explained that unlike plastic (which has white specular highlights), metals have coloured specular highlights that match their surface colour.
- The Final Implementation: We discarded the AI's PBR functions and wrote a custom mixing function: if uMetallic is high, the specular highlight tints the object's colour; if low, it remains white. This achieved the visual goal mathematically without needing the AI's complex texture setup.

```
// Specular: shiny highlights (metallic surfaces reflect their own color)
vec3 specColor = mix(vec3(1.0), uColor, uMetallic);
vec3 specular = specColor * (spec1 + spec2);
```

Example C: Dashboard Control Logic

- Context: We needed to connect the HTML sliders to the WebGL gripper variables.
- My Prompt: "Write a JavaScript event listener to update the gripper angle when a slider is moved."
- The AI Output (Functionally Correct): The AI provided a nearly perfect solution. It correctly identified that `oninput` (not `onchange`) was required for smooth, real-time animation, and it correctly targeted the specific array index.

```
document.getElementById("slider4").oninput = function(e) {  
    var newAngle = parseFloat(e.target.value);  
    theta[GRIPPER] = newAngle;  
    document.getElementById("v4").textContent = Math.round(newAngle) + "°";  
    updateUI();  
};
```

- Observations & Research:
 - Issue: While the code worked perfectly for moving the fingers, the gripper had no "physics". It would simply clip through the object without picking it up.
 - Research: We found an **MDN Web Docs** for "2D collision detection" [12] to find the simplest maths for checking proximity. We found the standard Euclidean distance formula being used for circular collision triggers.
- The Final Implementation: We kept the AI's suggested event structure but injected a single conditional block inside it. We added `'if (dist < 0.4) objectAttached = true;'`. This small addition changed the code from a simple animation script into a functional interaction.

```
function getGripperDistanceToCube() {  
    var tipPos = gripperWorldPos;  
    var dx = tipPos.x - cubePosition.x;  
    var dy = tipPos.y - cubePosition.y;  
    var dz = tipPos.z - cubePosition.z;  
  
    return Math.sqrt(dx*dx + dy*dy + dz*dz);  
}
```

```
if (wasOpen && !isNowOpen && !objectAttached) {  
    var dist = getGripperDistanceToCube();  
    if (dist < 0.4) {  
        objectAttached = true;  
    }  
}
```


5.3 Critical Analysis & Intellectual Contribution

While AI tools accelerated the generation of boilerplate code, most of the core simulation logic was made by us manually.

- Verification: We manually derived the matrix transformation order (Scale to Rotate to Translate) because the AI suggestions often reversed this, which caused the arm segments to rotate around the world origin rather than their local joints.
- Integration: The "Magnetic Grip" logic shown in Example C, where the object snaps to the gripper based on a distance threshold, was entirely custom logic we made to remove the need for a complex physics engine.

At times when we ran into errors we could not solve on our own, we looked to the AI tools to help debug the errors, which it did so correctly most of the time. We also made sure that we understood the errors we ran into, which ranged from using incorrect variables and data types to incorrect calculations.

5.4 External Code Citations

The following code blocks and mathematical concepts were adapted from open-source educational resources:

- Matrix & Vector Library (MV.js equivalent): The vector math functions (mult, translate, rotate, ortho) are standard implementations adapted from the textbook Interactive Computer Graphics by Angel & Shreiner[13].
- Phong Lighting Model: The vertex/fragment shader structure for per-pixel lighting is based on the MDN WebGL Lighting tutorial[14].

6.0 CONCLUSION

Through this project, we have successfully developed a fully functional 3D robotic arm “Pick and Place” digital twin using WebGL. Using hierarchical modelling, we have created a multi-jointed robotic arm that can be manually controlled. Rotational adjustments can be applied on the base, lower arm, upper arm, wrist and gripper. The color of the arm can also be changed. There is also an animation sequence that automates the relocation of the object from one platform to another, showcasing “Pick and Place”. The animation speed and looping can also be adjusted via the user’s interface controls. In short, the robotic arm is highly interactive, thus turning the static graphical visual representation into a parametric tool.

6.1 Success Factors and Validation

The technical success of the project relies on several key implementations:

- **Hierarchical Matrix Transformation:** The robotic arm joints have a parent-child relationship among them. Using nesting transformations, the children matrices will perform multiplication with the parent matrices to get global coordinates, allowing children components to move together with the parent. This matches the actual robotic arm operations, preventing the arm from breaking when rotating.
- **Lighting Model:** The Phong Lighting Model is implemented on the virtual simulation. The specular reflection term applied onto the robotic arm, the object and the platform allows them to appear somewhat shiny. The rim lighting also allowed the user to view and perceive the curvature of the cylindrical joints of the robotic arm.
- **Smooth Animation:** The animation sequence consists of 8 animation states. By using ease-in-out interpolation, the robotic arm is able to accelerate at the beginning and decelerate at the end of each state, achieving smooth and natural mechanical movement. The state machine logic also ensures the arm identifies the space to grab or drop the object.

6.2 Limitations and Challenges

However, there are distinct limitations in our current implementation:

- **Static Camera View:** The virtual simulation uses a fixed camera angle. Users, mainly developers, are not able to rotate or zoom on specific joints to inspect the arm movements. This makes it hard to validate the robotic arm functionalities from only a single perspective.
- **Lack of Inverse Kinematics:** The system currently uses forward kinematics, where it only allows users to toggle the angle of the robotic arm joints using the slider. The system does not have the click feature that allows users to choose a 3D coordinate, where the system will then calculate the required joint angles and perform the arm movement.
- **Lack of Shadow Mapping:** Neither the robotic arm nor the object platforms have a real-time shadow displayed on the ground plane. This reduces the immersion of the robotic arm in the 3D environment.

6.3 Future Improvements

To bring this project closer to professional standards, we have proposed the following steps:

- **Interactive Camera:** We would add a mouse-based control logic, allowing users to click and drag the 3D space to rotate the virtual world to view the many angles of the robotic arm.
- **Implement Inverse Kinematics:** We would implement complex calculation logic that would allow users to click on a place in the 3D space for the arm to move forward to.
- **Implement Shadow Maps:** We would implement shadow mapping to allow the arm and platforms to have shadows in the 3D environment.

In conclusion, by combining hierarchical modelling, shading and control logic, we have created a platform for simulating robotic movement. The project has successfully demonstrated the core principles and application of interactive computer graphics on computer infrastructure.

7.0 REFERENCES

- [1] Proven Robotics, "Pick and place automation equipment: Key benefits today - Proven Robotics," *Proven Robotics*, Sep. 23, 2025.
<https://provenrobotics.ai/pick-and-place-automation-equipment/>
- [2] Plan, S, "Robotic Arm Challenges - Dorna Robotic's Solutions," *Dorna Robotics*, Jan. 10, 2025.
<https://dorna.ai/blog/robotic-arm-challenges-and-solutions/>
- [3] Gallagher, N., & Armstrong, M. M., "Digital Twin," *IBM*, Nov. 17, 2025.
<https://www.ibm.com/think/topics/digital-twin>
- [4] Nvidia Glossary, "What is a digital twin? - NVIDIA Glossary," *NVIDIA*, n. d..
<https://www.nvidia.com/en-us/glossary/digital-twin/>
- [5] K. Shoemake, "Animating Rotation with Quaternion Curves," *Proceedings of SIGGRAPH*, 1985.
- [6] N. I. Badler, B. A. Barsky, and D. Zeltzer, "Making Them Move: Mechanics, Control, and Animation of Articulated Figures," *Morgan Kaufmann*, n. d..
- [7] MIT OpenCourseWare, "Computer Graphics – Hierarchical Modeling," *Course Notes*, n. d..
- [8] K. Wang et al., "Computer Graphics Simulation Systems for Robot Manipulators," *Robotics and Computer-Integrated Manufacturing*, 1989.
- [9] J. J. Craig, "Introduction to Robotics: Mechanics and Control," *Pearson Education*, n. D..
- [10] Stack Overflow, "Does flat shading require this much vertex duplication?" Stack Overflow, n.d. <https://stackoverflow.com/questions/>
- [11]Graphics - Realistic metal shader - Game Development Stack Exchange, n.d.
<https://gamedev.stackexchange.com/questions/>

[12] MDN Web Docs, "2D collision detection," MDN Web Docs, n.d.
https://developer.mozilla.org/en-US/docs/Games/Techniques/2D_collision_detection

[13] E. Angel and D. Shreiner, "Interactive Computer Graphics," University of New Mexico, CS 433 Resources, n.d. <https://www.cs.unm.edu/~angel/>

[14] MDN Web Docs, "Lighting in WebGL," MDN Web Docs, n.d.
https://developer.mozilla.org/en-US/docs/Web/API/WebGL_API/Tutorial/Lighting_in WebGL